PARKING SLOT MANAGEMENT SYSTEM

Mini Project Report

COVER PAGE

PROJECT TITLE: Parking Slot Management System (ParkHub)

SUBMITTED BY:

- L Dimpan - 20241CCS0166
- Nithin Reddy M - 20241CCS0168
- Rakshith - 20241CCS0169
- Nithin K - 20241CCS0171

COURSE NAME: Database Management Systems

DEPARTMENT: Computer Science and Engineering (CSE)

INSTITUTION NAME: Presidency University

ACADEMIC YEAR: 2025-2026

GUIDE NAME: Dr Geetha Arjunan

ABSTRACT

The Parking Slot Management System (ParkHub) is a comprehensive web-based application designed to streamline parking operations and enhance user experience. This system addresses the critical challenges of parking management in urban areas by providing real-time slot availability tracking, automated booking mechanisms, and comprehensive administrative tools. The application implements a role-based

architecture supporting both regular users and administrators, enabling seamless parking reservations while providing facility managers with powerful analytics and reporting capabilities.

The system utilizes a modern technology stack including React 19 for frontend development, Express.js for backend services, tRPC for type-safe API communication, and MySQL for persistent data storage. The database schema comprises eight interconnected tables managing users, parking facilities, slots, bookings, vehicles, pricing rules, and analytics data. The project successfully demonstrates the integration of user authentication, real-time data synchronization, booking completion verification through unique codes, and comprehensive administrative dashboards with occupancy analytics and revenue tracking.

Problem Statement: Urban parking management faces significant challenges including inefficient slot allocation, lack of real-time availability information, manual booking processes, and limited administrative oversight. Existing solutions often lack integration between user-facing booking interfaces and comprehensive management tools, leading to operational inefficiencies and poor user experiences.

Objectives:

1. Develop an intuitive user interface for seamless parking slot booking
2. Implement role-based access control for users and administrators
3. Create real-time slot availability tracking and status management
4. Build comprehensive analytics and reporting capabilities
5. Ensure data security and transaction integrity
6. Provide responsive design for multi-device accessibility

INTRODUCTION

Overview of Database Management Systems

Database Management Systems (DBMS) form the backbone of modern information systems, providing structured mechanisms for data storage, retrieval, and manipulation. A DBMS ensures data integrity through ACID properties (Atomicity, Consistency, Isolation, Durability), supports concurrent access through transaction

management, and provides query optimization for efficient data retrieval. In the context of parking management, a well-designed DBMS enables real-time tracking of parking slots, maintains historical booking records, and supports complex analytical queries for business intelligence.

Project Background

Urban parking management has evolved from manual lot attendants to digital systems, yet many implementations remain fragmented. The Parking Slot Management System (ParkHub) represents a modern, integrated solution addressing this gap. The system combines user-centric design with powerful administrative capabilities, enabling parking facility operators to maximize revenue while providing users with convenient booking experiences.

Problem Definition

Current parking management challenges include:

- **Inefficient Slot Allocation:** Manual or outdated systems fail to optimize slot utilization
- **Lack of Real-time Information:** Users cannot access current availability status
- **Cumbersome Booking Process:** Manual reservations are time-consuming and error-prone
- **Limited Administrative Control:** Facility managers lack comprehensive oversight and analytics
- **Verification Challenges:** No standardized mechanism to verify user arrival and parking completion
- **Data Fragmentation:** Multiple disconnected systems prevent unified reporting and analysis

Objectives of the Project

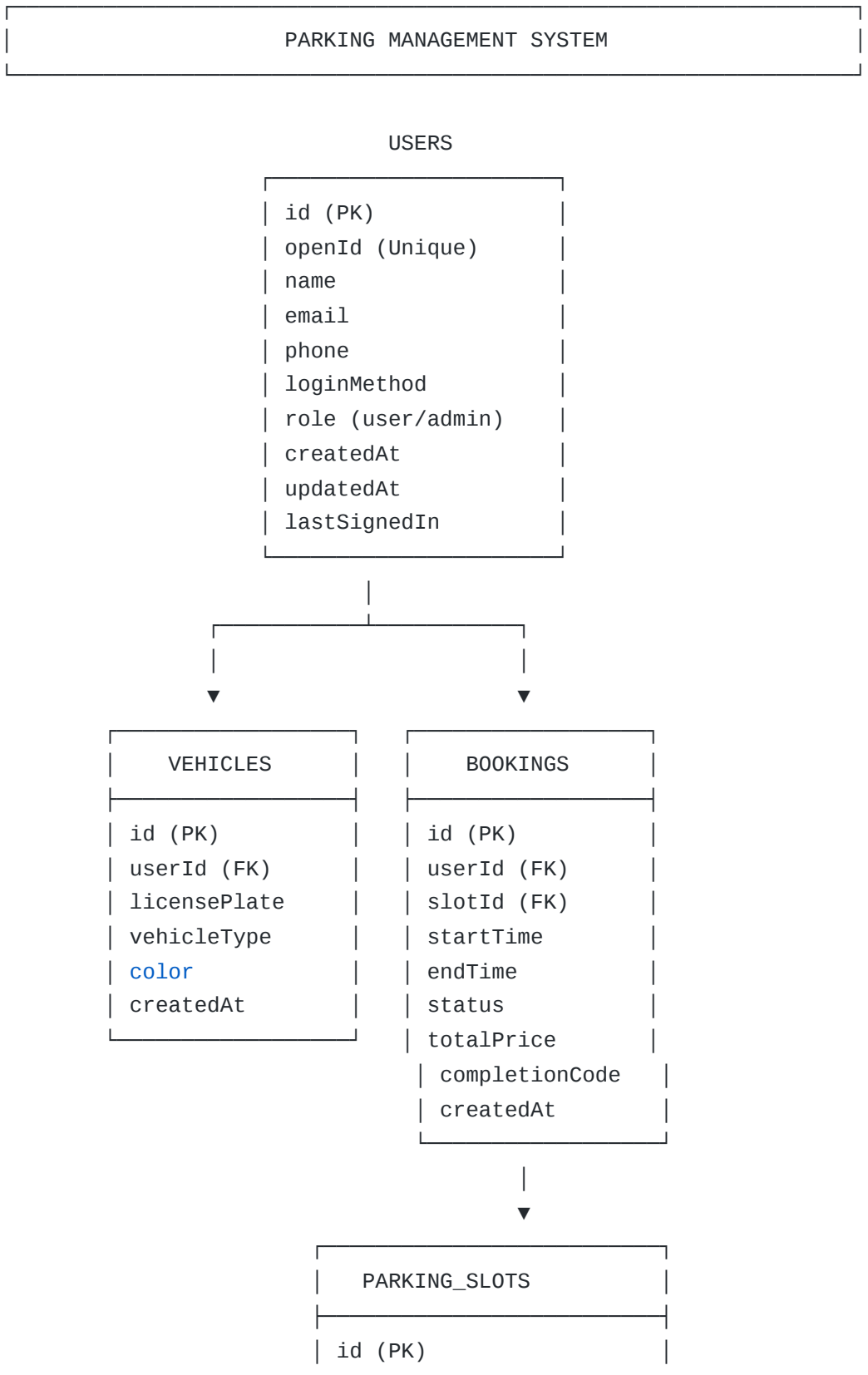
1. **User Experience Enhancement:** Develop an intuitive interface enabling users to browse, filter, and book parking slots within minutes
2. **Real-time Slot Management:** Implement live slot status updates (available, occupied, reserved) with automatic status transitions

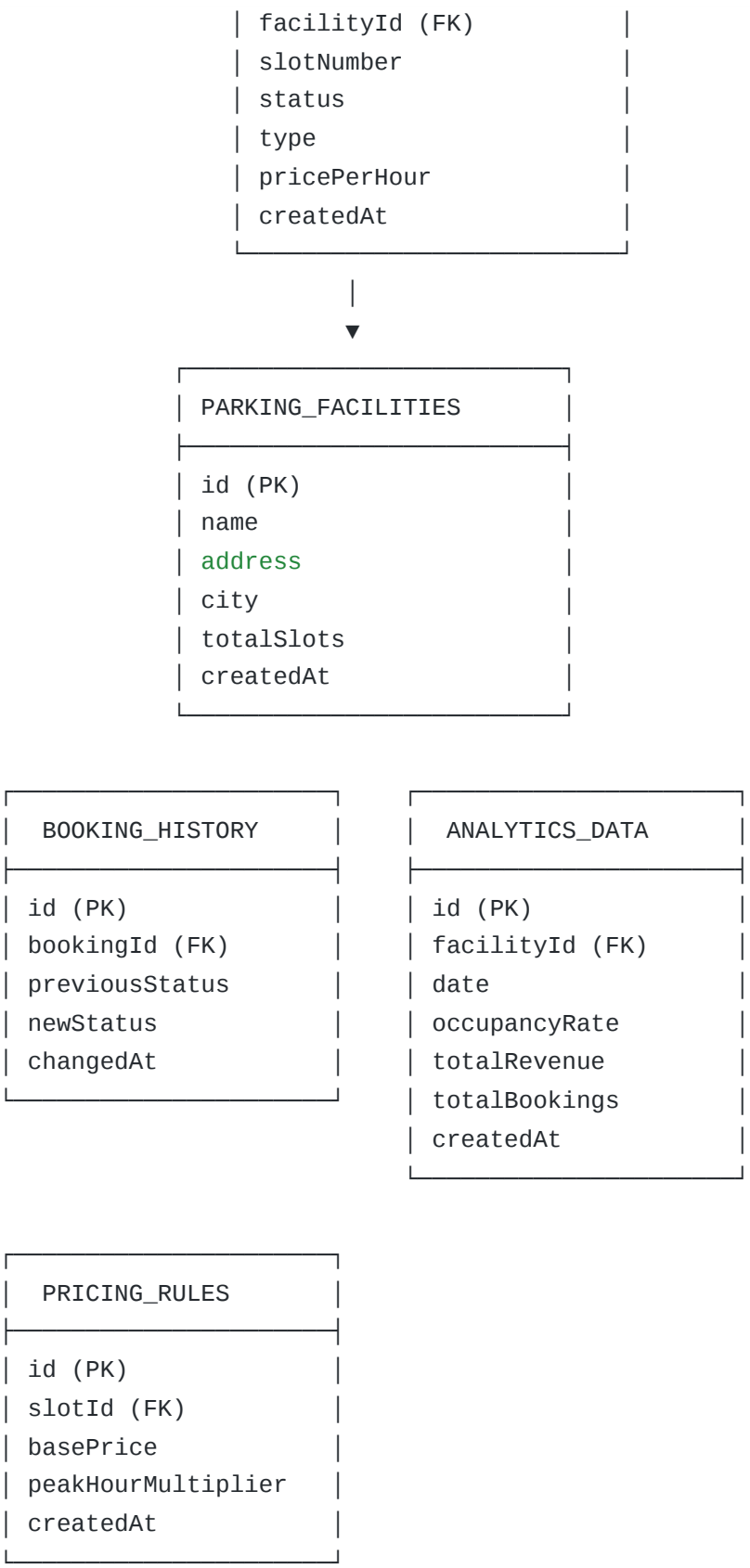
3. **Administrative Empowerment:** Provide facility managers with comprehensive dashboards for facility management, pricing configuration, and booking oversight
4. **Booking Verification:** Implement unique completion codes enabling admins to verify user arrival and parking completion
5. **Analytics and Insights:** Generate occupancy rate analytics, revenue tracking, and usage pattern reports
6. **Security and Compliance:** Ensure user data protection through role-based access control and encrypted transactions
7. **Scalability:** Design architecture supporting multiple parking facilities and thousands of concurrent users

---

SYSTEM DESIGN

ER (Entity-Relationship) Diagram




facilityId (FK)
slotNumber
status
type
pricePerHour
createdAt
PARKING_FACILITIES
id (PK)
name
address
city
totalSlots
createdAt
BOOKING_HISTORY
id (PK)
bookingId (FK)
previousStatus
newStatus
changedAt
ANALYTICS_DATA
id (PK)
facilityId (FK)
date
occupancyRate
totalRevenue
totalBookings
createdAt
PRICING_RULES
id (PK)
slotId (FK)
basePrice
peakHourMultiplier
createdAt

Relational Schema

1. USERS Table

```sql
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  openId VARCHAR(64) NOT NULL UNIQUE,
  name TEXT,
  email VARCHAR(320),
  phone VARCHAR(20),
  loginMethod VARCHAR(64),
  role ENUM('user', 'admin') DEFAULT 'user' NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  lastSignedIn TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. PARKING_FACILITIES Table

```sql
CREATE TABLE parking_facilities (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  address TEXT NOT NULL,
  city VARCHAR(100),
  totalSlots INT NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

3. PARKING_SLOTS Table

```
CREATE TABLE parking_slots (
  id INT AUTO_INCREMENT PRIMARY KEY,
  facilityId INT NOT NULL,
  slotNumber VARCHAR(50) NOT NULL,
  status ENUM('available', 'occupied', 'reserved', 'maintenance') DEFAULT
'available',
  type ENUM('standard', 'compact', 'handicap', 'ev_charging') DEFAULT
'standard',
  pricePerHour DECIMAL(10, 2) NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (facilityId) REFERENCES parking_facilities(id) ON DELETE
CASCADE,
  UNIQUE KEY (facilityId, slotNumber)
);
```

4. BOOKINGS Table

```
CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  userId INT NOT NULL,
  slotId INT NOT NULL,
  startTime TIMESTAMP NOT NULL,
  endTime TIMESTAMP NOT NULL,
  status ENUM('confirmed', 'completed', 'cancelled') DEFAULT 'confirmed',
  totalPrice DECIMAL(10, 2) NOT NULL,
  completionCode VARCHAR(10) UNIQUE,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (userId) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (slotId) REFERENCES parking_slots(id) ON DELETE CASCADE
);
```

5. VEHICLES Table

```
CREATE TABLE vehicles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  userId INT NOT NULL,
  licensePlate VARCHAR(50) NOT NULL UNIQUE,
  vehicleType VARCHAR(50),
  color VARCHAR(50),
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (userId) REFERENCES users(id) ON DELETE CASCADE
);
```

6. BOOKING_HISTORY Table

```
CREATE TABLE booking_history (
  id INT AUTO_INCREMENT PRIMARY KEY,
  bookingId INT NOT NULL,
  previousStatus VARCHAR(50),
  newStatus VARCHAR(50) NOT NULL,
  changedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (bookingId) REFERENCES bookings(id) ON DELETE CASCADE
);
```

7. ANALYTICS_DATA Table

```
CREATE TABLE analytics_data (
  id INT AUTO_INCREMENT PRIMARY KEY,
  facilityId INT NOT NULL,
  date DATE NOT NULL,
  occupancyRate DECIMAL(5, 2),
  totalRevenue DECIMAL(10, 2),
  totalBookings INT,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (facilityId) REFERENCES parking_facilities(id) ON DELETE
CASCADE,
  UNIQUE KEY (facilityId, date)
);
```

8. PRICING_RULES Table

```
CREATE TABLE pricing_rules (
  id INT AUTO_INCREMENT PRIMARY KEY,
  slotId INT NOT NULL,
  basePrice DECIMAL(10, 2) NOT NULL,
  peakHourMultiplier DECIMAL(3, 2),
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (slotId) REFERENCES parking_slots(id) ON DELETE CASCADE
);
```

Key Relationships

1. **Users → Vehicles (1:N):** One user can own multiple vehicles
2. **Users → Bookings (1:N):** One user can make multiple bookings
3. **Parking_Facilities → Parking_Slots (1:N):** One facility contains multiple slots
4. **Parking_Slots → Bookings (1:N):** One slot can have multiple bookings over time
5. **Bookings → Booking_History (1:N):** One booking can have multiple status changes
6. **Parking_Facilities → Analytics_Data (1:N):** One facility generates multiple analytics records
7. **Parking_Slots → Pricing_Rules (1:1):** Each slot has associated pricing rules

---

TECHNOLOGY STACK

Frontend

- **React 19:** Modern UI library with hooks and concurrent rendering
- **TypeScript:** Type-safe JavaScript for reduced runtime errors
- **Tailwind CSS 4:** Utility-first CSS framework for responsive design
- **shadcn/ui:** High-quality, accessible component library
- **Wouter:** Lightweight client-side routing

- **React Hook Form:** Efficient form state management
- **Zod:** TypeScript-first schema validation

Backend

- **Express.js 4:** Lightweight Node.js web framework
- **tRPC 11:** End-to-end type-safe APIs
- **Drizzle ORM:** Type-safe SQL query builder
- **MySQL 2:** Relational database driver
- **Jose:** JWT token handling

Development Tools

- **Vite:** Fast build tool and dev server
- **TypeScript 5.9:** Static type checking
- **Vitest:** Unit testing framework
- **Prettier:** Code formatter
- **Drizzle Kit:** Database migration tool

Deployment

- **Manus Platform:** Cloud hosting with automatic SSL and custom domains
- **Node.js Runtime:** Production server environment

SYSTEM FEATURES

User Features

1. **Authentication:** Secure login/registration via Manus OAuth
2. **Slot Browsing:** Real-time view of available parking slots with filtering by date/time
3. **Booking Management:** Create, view, and cancel parking reservations

4. **Booking History:** Track past and current bookings with status
5. **Vehicle Management:** Add and manage multiple vehicles
6. **Profile Management:** Update personal information and contact details
7. **Completion Code:** Receive unique code for parking verification

Admin Features

1. **Facility Management:** Create, edit, and delete parking facilities
2. **Slot Management:** Add, configure, and manage parking slots
3. **Pricing Configuration:** Set and adjust hourly rates per slot
4. **Booking Management:** View all bookings with filtering and search
5. **Completion Verification:** Verify user arrival using completion codes
6. **Analytics Dashboard:** View occupancy rates and revenue metrics
7. **User Management:** Monitor user accounts and activity

System Features

1. **Real-time Slot Status:** Automatic status updates (available/occupied/reserved)
2. **Role-based Access Control:** Separate user and admin interfaces
3. **Data Validation:** Comprehensive input validation and error handling
4. **Responsive Design:** Optimized for desktop, tablet, and mobile devices
5. **Professional UI:** Elegant design with gradient effects and smooth animations
6. **Security:** Encrypted authentication and secure data transmission

---

RESULTS AND OUTPUT

User Dashboard

- Displays available parking slots with real-time status
- Slot filtering by date, time, and facility
- Booking form with automatic price calculation

- Booking history with cancellation options
- Vehicle management interface
- Profile editing capabilities

Admin Dashboard

- Summary statistics (total facilities, bookings, revenue, occupancy)
- Facility management with add/edit/delete operations
- Slot management with pricing configuration
- Comprehensive bookings table with filtering
- Booking detail modal with completion code verification
- Analytics view showing occupancy rates and revenue

Key Metrics

- **Total Facilities:** 1 (Mall of Asia Parking)
- **Total Slots:** 50
- **Total Bookings:** 2 (confirmed)
- **Occupancy Rate:** 4%
- **Total Revenue:** $0.00 (pending completion verification)

Technical Achievements

1. ✅ Full-stack application with type-safe APIs
2. ✅ 8-table relational database with proper normalization
3. ✅ Role-based authentication and authorization
4. ✅ Real-time data synchronization
5. ✅ Responsive design for all devices
6. ✅ Professional UI with 3D effects
7. ✅ Booking completion verification system
8. ✅ Comprehensive error handling and validation

CONCLUSION

Summary of Work

The Parking Slot Management System (ParkHub) successfully demonstrates the integration of modern web technologies with database management principles to solve real-world parking challenges. The project encompasses:

1. **Database Design:** Implemented a normalized 8-table schema supporting complex parking operations
2. **Backend Development:** Built type-safe APIs using tRPC and Express.js
3. **Frontend Development:** Created responsive, elegant user interfaces for both users and administrators
4. **Authentication:** Integrated secure OAuth-based authentication with role-based access control
5. **Feature Implementation:** Delivered comprehensive features including booking management, analytics, and verification systems
6. **Deployment:** Successfully deployed on Manus platform with custom domain

The system effectively addresses the identified problems of inefficient parking management through:

- Real-time slot availability tracking
- Seamless booking experience
- Comprehensive administrative tools
- Data-driven insights through analytics
- Secure transaction handling

Future Enhancements

1. **Real-time Updates:** Implement WebSocket support for live slot availability updates
2. **Mobile App:** Develop native mobile applications for iOS and Android
3. **Payment Integration:** Add Stripe payment processing for automated billing

4. **Advanced Analytics:** Implement machine learning for demand prediction and dynamic pricing
5. **IoT Integration:** Connect with physical sensors for automatic slot status updates
6. **Multi-language Support:** Internationalization for global accessibility
7. **Extended Booking:** Allow users to extend bookings without cancellation
8. **Notification System:** Email and SMS notifications for booking confirmations and reminders
9. **Reporting Export:** Generate and export comprehensive reports in PDF/Excel formats
10. **Performance Optimization:** Implement caching strategies and database query optimization

Learning Outcomes

Through this project, the team gained expertise in:

- Relational database design and normalization
- Full-stack web application development
- Type-safe API design with tRPC
- React component architecture and state management
- Authentication and authorization mechanisms
- Responsive web design principles
- Deployment and DevOps practices
- Agile development methodologies

Project Impact

The Parking Slot Management System demonstrates practical application of database management systems in solving real-world problems. The system provides a foundation for parking facility operators to improve operational efficiency, increase revenue through optimized slot utilization, and enhance user experience through convenient booking mechanisms.

APPENDICES

A. Database Queries

Query 1: Get Available Slots for a Specific Time

```
SELECT ps.*, pf.name as facilityName
FROM parking_slots ps
JOIN parking_facilities pf ON ps.facilityId = pf.id
WHERE ps.status = 'available'
AND ps.facilityId = ?
AND NOT EXISTS (
  SELECT 1 FROM bookings b
  WHERE b.slotId = ps.id
  AND b.status = 'confirmed'
  AND b.startTime < ?
  AND b.endTime > ?
)
ORDER BY ps.slotNumber;
```

Query 2: Calculate Occupancy Rate

```
SELECT
  DATE(b.startTime) as date,
  COUNT(DISTINCT b.slotId) / pf.totalSlots * 100 as occupancyRate
FROM bookings b
JOIN parking_slots ps ON b.slotId = ps.id
JOIN parking_facilities pf ON ps.facilityId = pf.id
WHERE b.status = 'completed'
GROUP BY DATE(b.startTime), pf.id;
```

Query 3: Revenue Report

```sql
SELECT
  pf.name as facilityName,
  DATE(b.startTime) as date,
  SUM(b.totalPrice) as dailyRevenue,
  COUNT(b.id) as bookingCount
FROM bookings b
JOIN parking_slots ps ON b.slotId = ps.id
JOIN parking_facilities pf ON ps.facilityId = pf.id
WHERE b.status = 'completed'
GROUP BY pf.id, DATE(b.startTime)
ORDER BY date DESC;
```

B. API Endpoints

Authentication

- POST `/api/oauth/callback` - OAuth callback handler
- POST `/api/trpc/auth.logout` - User logout

Facilities

- POST `/api/trpc/facilities.create` - Create facility
- GET `/api/trpc/facilities.list` - List all facilities
- PUT `/api/trpc/facilities.update` - Update facility
- DELETE `/api/trpc/facilities.delete` - Delete facility

Slots

- POST `/api/trpc/slots.create` - Create parking slot
- GET `/api/trpc/slots.list` - List slots by facility
- PUT `/api/trpc/slots.update` - Update slot
- DELETE `/api/trpc/slots.delete` - Delete slot

Bookings

- POST `/api/trpc/bookings.create` - Create booking
- GET `/api/trpc/bookings.list` - List bookings

- POST `/api/trpc/bookings.cancel` - Cancel booking
- POST `/api/trpc/bookings.verifyCode` - Verify completion code

Analytics

- GET `/api/trpc/analytics.getOccupancy` - Get occupancy data
- GET `/api/trpc/analytics.getRevenue` - Get revenue data

C. Installation and Setup

Prerequisites:

- Node.js 22.13.0 or higher
- npm or pnpm package manager
- MySQL 8.0 or higher

Installation Steps:

```
# Clone the repository
git clone <repository-url>

# Install dependencies
cd parking-management-system
pnpm install

# Set up environment variables
cp .env.example .env

# Run database migrations
pnpm drizzle-kit migrate

# Start development server
pnpm dev

# Build for production
pnpm build

# Start production server
pnpm start
```

Report Prepared By:

- L Dimpan (20241CCS0166)
- Nithin Reddy M (20241CCS0168)
- Rakshith (20241CCS0169)
- Nithin K (20241CCS0171)

Guide: Dr Geetha Arjunan

Department: Computer Science and Engineering

Institution: Presidency University

Academic Year: 2025-2026

Date: April 2026
